

Group 5: Finance Tracker Database

Aila Chin, Keanna Dirga, Angel Ibanez, Jonathan Piccoli, Jeena Weber Langstaff

May 6, 2026

A. Application Overview

Our application is a Finance Tracker Database designed to help users organize and review their personal financial information. It stores data about account holders, financial accounts, transactions, categories, and payees or sources. By using relationships to connect tables and information, users can keep track of income, expenses, account activity, and spending patterns all in one place.

It was made to be a simple and structured way to monitor and review a user's personal finances. Each person can have multiple accounts with multiple different bills, purchases, and sources of income that can make it difficult to understand where their money is going. With Finance Tracker, users can store their financial information in a way that is easily accessible and can be searched, filtered, and analyzed. Due to it not connecting directly to real bank accounts, the user fills in their own information. This lets the database act as a personal finance log while avoiding direct access to sensitive banking systems.

The target audience is anyone who wants an easier way to view and track their finances. This can be students, young adults, families, or anyone who wants to track income, expenses, bills, debt, and spending habits. It is also useful for anyone who wants to review their financial activity without relying only on bank statements or separate budgeting tools.

The key features are its organized database structure and its ability to connect related financial information. Users can be connected to one or more accounts, and each account can have many transactions. Transactions can also be labeled in categories such as groceries, rent, income, or utilities. These can then be connected to a payee or source like a store, company, or employer. This makes it easy to query spending habits, compare categories, review financial data, and see where money comes and goes. The database design also separates account holders, accounts, transactions, categories, and payee/sources so financial data is easier to manage and less repetitive.

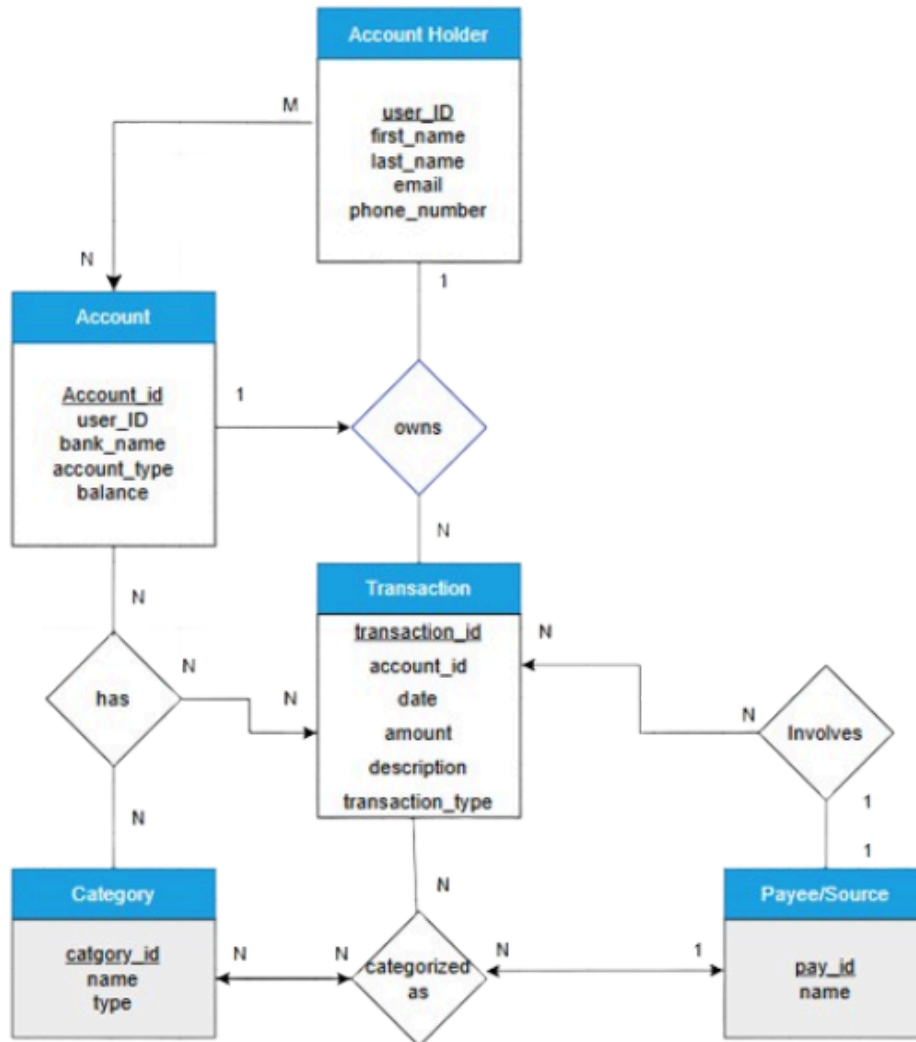
B. Database Design

The E-R Diagram for this database includes 5 entities and 4 relationships. We have 3 primary entities such as Account Holder, Account, and Transaction. Account Holder represents the individual person who owns the specific bank account we are looking at, and Transaction represents all of the different transactions within that Account entity. This entity includes 5 attributes, with the primary key being user_ID. This user_ID attribute serves as the main identifier for the user, and must be unique so that no two people will hold the same ID. The main constraint for the primary key is that it must be an integer, and as a new ID is created, the numbers will increase incrementally. Within the other 5 attributes, we have 'Email' which also must be unique. This attribute has a constraint of varchar(50), and because we use Email as our

primary source of contact for the user, this allows our Phone_Number entity to be null if the user chooses. The Account Holder entity has a relationship with our Account entity with a 'Many to One' connection. Each account can have one account holder, but an account holder can have multiple accounts.

The Account entity uses the User_ID from the Account Holder entity as a foreign key to form the direct relationship. The primary key for Account is Account_ID and holds the same constraints as user_ID. It must be int(11), not null, and unique. This entity represents the bank account that the user holds. It includes information such as bank_name, account_type, and balance. Account_type only has three options: Money Market, Savings, and Checking. This simplifies organization because it categorizes the types of accounts. Balance has a format of decimal(10,2) because it must take a currency format with two decimal points and we decided on a total integer amount of 10 for this database, because it is very rare for a user to perform transactions with larger numbers. In this entity, Account_ID serves as a foreign key for our next entity, 'Transaction'. With our E-R diagram, we see that each Transaction can only be connected to one account and account holder, but there can be multiple transactions within these accounts. Account Holder has a relationship with Account using a foreign key, it has a relationship with Transaction as well.

This Transaction entity serves as our most active table, because it is the heart of our database. There will be many different transactions for each account and account holder. We want our primary key, Transaction_ID, to be unique and also increment as each new transaction is made. It takes the Account_ID in order to keep the relationship and identify where the transaction is coming from, and also holds the date, amount, transaction_type, and description as attributes. For the date of the transaction, we decided to format it as year-day-month for filtering and visual purposes. Transaction_Type, will only have two options: withdraw and deposit. This is useful because we wanted to keep the amount a positive number, so we use the transaction type to determine if it is taking or putting money into the account. The description for the transaction comes from a small entity named 'Category.' Category has its own Category_ID which is the primary key and holds similar constraints as the other ID primary keys. The type attribute only has two outcomes: income and expense, which come hand in hand with the Transaction_Type. The name attribute of the category entity holds the information for Transaction regarding the description. The Category_ID number will link to different values in the type attribute which gives the description for Transaction. For example, Category_ID = 1 will hold a type of 'Salary', while Category_ID=2 will hold a type of 'Groceries.'



C. Database Implementation

The integration from our E-R Diagram to MySQL was very simple, but we did have to make a few tweaks to our tables in order to have our relationship run smoothly. The database was populated with information generated from ChatGPT and then imported into an Excel spreadsheet. We applied constraints and filtration with the formula tab in Excel. For example, in order to create an amount value based off of Category_ID which is column C2, we use an 'If' statement to randomly generate a value. Example below:

```
=IF(C2=1,RANDBETWEEN(100,4000),
IF(C2=2,RANDBETWEEN(1,700),
IF(C2=3,RANDBETWEEN(750,4000),
IF(C2=4,RANDBETWEEN(1,5000),
IF(C2=5,RANDBETWEEN(1,200),
IF(C2=6,RANDBETWEEN(1,5000),
IF(C2=7,RANDBETWEEN(100,600),
IF(C2=8,RANDBETWEEN(2,100),
IF(C2=9,RANDBETWEEN(300,10000),
IF(C2=10,RANDBETWEEN(100,600),
""))))))))))
```

```
=IF(C2=1,"Salary",
IF(C2=2,"Groceries",
IF(C2=3,"Rent",
IF(C2=4,"Dropshipping",
IF(C2=5,"Games",
IF(C2=6,"Misc",
IF(C2=7,"Utilities",
IF(C2=8,"Transportation",
IF(C2=9,"Education",
IF(C2=10,"Insurance",
""))))))))))
```

Category_ID	Name	Type
1	Salary	Income
2	Groceries	Expense
3	Rent	Expense
4	Dropshipping	Income
5	Games	Expense
6	Miscellaneous	Expense
7	Utilities	Expense
8	Transportation	Expense
9	Education	Expense
10	Insurance	Expense

1	50	4	7	2026-01-11	1536	Dropshipping	Deposit
2	96	6	7	2025-12-02	1263	Misc	Withdrawal
3	82	4	7	2026-01-16	4888	Dropshipping	Deposit
4	38	2	5	2025-12-31	110	Groceries	Withdrawal
5	45	7	3	2025-12-03	377	Utilities	Withdrawal
6	22	2	5	2026-01-06	225	Groceries	Withdrawal
7	26	3	4	2026-01-21	1809	Rent	Withdrawal

These constraints and formulas helped ensure consistency and valid data entry throughout the system within our first step of populating our dataset. The Excel sheet mirrored the tables we created in MySQL, so importing it was simple since the column order was identical. The primary keys from our entity tables were kept the same for the MySQL database, however we altered our foreign keys a bit. For our Transaction Table, we have 1 primary key and 3 foreign keys while in our E-R Diagram, we only had one foreign key 'account_ID.' In our newly created table, we have Account_ID, Category_ID, and Pay_ID as foreign keys. All of these foreign keys were primary keys of their respective tables in order to ensure that our Transaction table had

relationships with every table it needed information from. The Transaction table used Account_ID in order to check the User_ID (who made the transaction), Bank_Name, Account_Type, and Balance. Category_ID was used in order to grab the name of the category and whether the transaction was an income or expense. Pay_ID was used in order to check where the transaction was made. Using ChatGPT to generate data, Excel to organize the data, and then a general cleanup of our E-R diagram and relationship logic, we were able to create a fully functional database for our finance tracker.

								Category_ID	Name	Type
								1	Salary	Income
								2	Groceries	Expense
								3	Rent	Expense
								4	Dropshipping	Income
								5	Games	Expense
								6	Miscellaneous	Expense
								7	Utilities	Expense
								8	Transportation	Expense
								9	Education	Expense
								10	Insurance	Expense
Transaction_ID	Account_ID	Category_ID	Pay_ID	date	amount	description	transaction_type			
1	50	4	7	2026-01-11	1536.00	Dropshipping	Deposit			
2	96	6	7	2025-12-02	1263.00	Misc	Withdrawal			
3	82	4	7	2026-01-16	4888.00	Dropshipping	Deposit			
4	38	2	5	2025-12-31	110.00	Groceries	Withdrawal			
5	45	7	3	2025-12-03	377.00	Utilities	Withdrawal			
6	22	2	5	2026-01-06	225.00	Groceries	Withdrawal			
7	26	3	4	2026-01-21	1809.00	Rent	Withdrawal			
8	29	3	4	2025-09-02	1574.00	Rent	Withdrawal			
9	85	2	2	2025-09-07	222.00	Groceries	Withdrawal			
10	94	2	5	2025-12-01	602.00	Groceries	Withdrawal			
11	13	6	8	2026-01-04	2949.00	Misc	Withdrawal			
Account_ID	User_ID	Bank_name	Account_type	Balance						
3	3	Capital One	Money Market	2172.70	User_ID	First_Name	Last_Name	Email	Phone_number	
4	4	Truist	Savings	516.01	1	Benjamin	Jackson	benjamin.jackson1@example.com	326381088	
5	5	TD Bank	Checking	3700.98	2	Emma	Taylor	emma.taylor2@example.com	+1-999-9999	
6	6	Bank of America	Money Market	2043.64	3	Ava	Moore	ava.moore3@example.com	1626214580	
7	7	Wells Fargo	Checking	4494.09	4	Mia	Hernandez	mia.hernandez4@example.com	827498608	
8	8	U.S. Bank	Money Market	2919.90	5	Benjamin	Moore	benjamin.moore5@example.com	961577524	
9	9	Wells Fargo	Savings	1610.19	6	Harper	Anderson	harper.anderson6@example.com	415554534	
10	10	U.S. Bank	Money Market	1441.39	7	Ava	Thomas	ava.thomas7@example.com	974779629	
11	11	Truist	Savings	2026.33	8	Benjamin	Johnson	benjamin.johnson8@example.com	567795656	
12	12	Capital One	Savings	835.42	9	Noah	Martin	noah.martin9@example.com	1199700961	
13	13	Citibank	Checking	1715.38	10	Harper	Rodriguez	harper.rodriguez10@example.com	2111737782	
14	14	Wells Fargo	Money Market	3789.49	11	James	Martin	james.martin11@example.com	1156118306	
					12	Mia	Moore	mia.moore12@example.com	1316781548	

D. Normalization

Our finance tracker satisfies 3NF by structuring our data so that it minimizes duplication, keeps related information in the right place, and ensures that everything depends only on the primary keys. Instead of storing repeated details such as account names, categories, or user information in every transaction record, our design separates these into their own tables. For example, instead of repeating “Checking” as the account type in every row of transaction, we store it once in the Account table and reference it using our foreign primary keys. We also made sure to organize the data into appropriate tables. The Finance Tracker breaks the data into

multiple tables such as Transactions, Accounts, Categories, Account_Holder, and PayeeSource with each table having its own purpose. Transaction stores the amount, dates, and references multiple tables with foreign keys. The 'Account' stores the bank account information such as bank name, amount, and account type. Account_Holder stores the information of the individual user such as their name, phone number, and mail address. The separation of tables helps ensure that each table represents a single concept which is the core idea of 3NF. The biggest improvement in transforming our database to 3NF is removing transitive dependencies. Instead of storing category names or account details directly in the Transaction table, we only store the IDs. Because of this, attributes like description depend on Category_ID from the Category table, not the Transaction_ID. Methods like these ensure that all non-key attributes depend only on the primary key of their own table. Overall, by separating tables, using foreign keys, and eliminating transitive dependencies, the Finance Tracker maintains consistent and clean data that is easier to update, query, and scale with minimal errors.

E. Application Functionality & Database Integration

Finance Tracker works as a database-driven system because the user interface is connected to a MySQL database that stores and organizes the application's financial data. The database has tables including Account_Holder, Category, Payee/Source, and Transactions to store user information, account details, transaction records, spending categories, and payee/source information. When a user views financial data, adds a transaction, filters information, or reviews account activity, the application sends a request to the backend, which uses SQL queries to retrieve and update the database.

This allows for easy support of meaningful financial tracking. Users can record income and expenses, link transactions to specific accounts, sort spending into specific categories, and review where money is coming or going. Because the data is stored in related tables, information can be combined from multiple parts of the database, such as showing a transaction with the account holder's name, the account that was used, the transaction category, and the payee or source. This makes the system more useful than a spreadsheet because the data can be easily searched, updated, and reviewed with queries.

The first important query is the user login authentication. The user enters their email address and phone number, and the application checks whether a matching record exists in the Account_Holder table

```
function authenticate(PDO $pdo, string $email, string $phone): array|false {
    $stmt = $pdo->prepare(
        'SELECT * FROM Account_Holder WHERE Email = :email AND Phone_number = :phone'
    );
    $stmt->execute([':email' => $email, ':phone' => $phone]);
    return $stmt->fetch();
}

<?php
session_start();
```

If the email and phone match a record in the database, the query returns that user's account holder information. The user's ID, first name and last name, and email are stored in the session. If there is no matching record the login fails and the user receives an error message.

This supports the application by connecting the login form to the database. It ensures that only users with matching account holder information can access the dashboard. The session file has authentication handled through the prepared SQL statement, which cannot be manipulated by bad actors.

The second query calculates the logged-in user's deposit total. A similar query is also used for withdrawals. These values are both displayed in the dashboard summary cards.

```
'SELECT COALESCE(SUM(t.amount), 0) AS total, COUNT(*) AS cnt
FROM Transactions t
JOIN Account a ON t.Account_ID = a.Account_ID
WHERE a.User_ID = :uid AND t.transaction_type = "Deposit"'
```

TOTAL BALANCE \$5,600.00 1 account	TOTAL DEPOSITS \$7,000.00 1 entry
TOTAL WITHDRAWALS \$1,500.00 1 entry	NET \$5,500.00 Surplus

This query returns the total dollar amount for deposits, and the number of deposit entries for the user. The dashboard uses these results to display both totals.

This supports the application by turning the raw transaction records into quick and useful financial summaries.

Instead of making users calculate deposits manually, the application uses SQL aggregate functions to summarize the user's financial data. The dashboard code also uses a matching query for withdrawals, then calculates the net amount by subtracting total withdrawals from total deposits. The final one displays the user's transaction history. It joins multiple tables together so the dashboard can show complete transaction information instead of only ID numbers.

```
SELECT t.Transaction_ID, t.date, t.amount, t.description, t.transaction_type,
       c.Name      AS category_name,
       p.Name      AS payee_name,
       a.Bank_name,
       a.Account_type
FROM Transactions t
JOIN Category      c ON t.Category_ID = c.Category_ID
JOIN `Payee/Source` p ON t.Pay_ID      = p.Pay_ID
JOIN Account       a ON t.Account_ID   = a.Account_ID
WHERE a.User_ID = :uid
```

This returns the user's transaction history with readable category names, payee/source names, bank names, and account types. The results are then displayed in the transaction table on the dashboard.

All Transactions							+ Add Transaction
All Types	All Categories	All Accounts	Search transactions...				
Date	Description	Category	Payee / Source	Account	Type	Amount	
Apr 29, 2026	Payday	Salary	Employer Inc	Chase - Savings	Deposit	+\$7,000.00	×
Apr 29, 2026	Bills	Utilities	Electric Company	Chase - Savings	Withdrawal	-\$1,500.00	×
Showing 2 of 2 transactions							

This query is one of the most important in the application because it connects the database tables to a readable interface. The database stores multiple IDs, but the user should be able to see the names and descriptions instead. Joins allow the application to display meaningful transaction data while keeping the database organized. The dashboard also expands the query

Each row includes the date, description, category, payee//source, account, transaction type, and amount. Depository and withdrawals are visually separated using different colors and labels, making it easier for users to understand their financial activity at a glance. The dashboard also includes filters for transaction type, category, account, and search terms, which helps users narrow down the list.

Add Account

Link a new bank account to your profile

Bank Name:

Final

Account Type:

Checking

Starting Balance (\$):

1000

Add Account

The Add Account page provides an easy form for linking a new bank account to the users profile. The user enters the bank name, account type, and starting balance.

Your Accounts

Bank	Type	Balance
Chase	Savings	\$10,650.00

After submitting, the page displays the user’s existing accounts below the form, allowing the user to confirm that the new account was created and added successfully.

Your Accounts

Bank	Type	Balance
Final	Checking	\$1,000.00
Chase	Savings	\$10,650.00

Overall, the user experience is straightforward, organized, and intuitive. Users can log in, view their dashboard, add accounts, add transactions, filter transaction records, and log out without needing to interact directly with the database. The interface hides the complexity of the SQL and presents the financial data in an easy to understand and readable format.

For frontend and backend integration, they are done through HTML forms, PHP processing, and MySQL queries. The frontend provides forms and buttons that allows users to input or request data. The backend receives the form data, validates the request, and runs SQL statements using PDO. the database then returns results or stores changes, and the website displays the updated information to the user.

Users input data through the login, registration, add account, and add transaction form. The login form sends the user's email and phone number to the backend, where PHP checks the Account_Holder table for a matching user. The registration form inserts a new account holder after checking whether the email already exists. The add account form inserts a new account into the Account table for the logged in user. The add transaction form inserts a new transaction into the Transactions table and updates the related account balance.

Log In

Email:

Phone Number:

Log In

Create Account

First Name:

Last Name:

Email:

Phone Number:

Create Account

Already have an account? [Log in](#)

New Transaction

Account: Final (Checking) **Date:** 05/06/2026

Amount (\$): 0.00 **Type:** Deposit

Category: Salary **Payee / Source:** Employer Inc

Description:
Brief note...

Cancel **Add Transaction**

Users retrieve data mainly through the dashboard. When the dashboard loads, PHP queries the database from the logged in user's accounts, total deposits, total withdrawals, and transactions. The transaction list is retrieved through a join query that combines information from Transactions, Category, Payee/Source, and Account. This allows the page to display complete transaction information rather than only foreign key values.

Data is managed through add and delete actions. When a user adds a new transaction, the backend inserts the transaction and updates the account balance. When a user deletes a transaction, the backend deletes it only if it belongs to an account connected to the logged in user. This helps prevent users from deleting transactions that don't belong to them.

The website uses HTML for page structure, CSS for styling, JavaScript for interface behavior such as toggling the user dropdown, PHP for backend logic, sessions for login state, PDO for database communication, and MySQL for data storage.

F. Security, Privacy & Data Protection

Our Finance Tracker implements multiple security measures across the application to protect user financial data from common web vulnerabilities. We took a defense-in-depth approach, implementing security at three levels: the database layer, the application layer, and the authentication layer.

SQL injection is a type of attack where malicious code is inserted into user input to manipulate database queries. We prevent this by using prepared statements with parameter binding. This approach separates the SQL code structure from user input data, ensuring the database treats user input as data only, never as executable code.

```
function authenticate(PDO $pdo, string $email, string $phone, string $password): array|false {  
  
    // Step 1: Query database with prepared statement  
  
    $stmt = $pdo->prepare(  
  
        'SELECT * FROM Account_Holder WHERE Email = :email AND Phone_number = :phone'  
  
    );  
  
    // Step 2: Execute with parameters safely separated from SQL  
  
    $stmt->execute([':email' => $email, ':phone' => $phone]);  
  
    // Step 3: Fetch user record if email and phone match  
  
    $user = $stmt->fetch();
```

```

// Step 4: Verify password against stored hash

if ($user && password_verify($password, $user['Password_Hash'])) {

    return $user; // All three credentials match

}

return false; // Email/phone not found or password incorrect

}

```

The Email and Phone are placeholders. Instead of mixing user input directly into the SQL command, we keep them separate. The database receives the command structure first, then receives the user's data as pure information, not written as code. Even if an attacker attempts SQL injection like ' OR ' 1 ' = ' 1, the database recognizes it as a literal value, not a SQL command. We use this same approach in all our queries. For example, when retrieving a user's deposit totals.

```

SELECT COALESCE(SUM(t.amount), 0) AS total, COUNT(*) AS cnt

FROM Transactions t

JOIN Account a ON t.Account_ID = a.Account_ID

WHERE a.User_ID = :uid AND t.transaction_type = "Deposit"

// "t" is just a short nickname so we can refer to it as "t." instead of "Transactions."

// a.User_ID = :uid means "only show transactions for this logged-in user" and :uid is a placeholder (safe
way to add the user's ID)

```

Cross-Site Scripting (XSS) attacks occur when malicious JavaScript code is injected into a web page. We prevent this using the htmlspecialchars() function, which encodes user-generated content before displaying it. This converts special characters into harmless HTML entities that display as text rather than executable code.

```

<p class="page-sub">Welcome back, <?= htmlspecialchars($_SESSION['first_name']) ?>!</p>

// htmlspecialchars() converts any < or > characters to safe HTML entities

// So malicious code displays as text instead of running

```

The SQL Table Code:

```
<td><?= htmlspecialchars($tx['description']) ?></td>
<td><?= htmlspecialchars($tx['category_name']) ?></td>
<td><?= htmlspecialchars($tx['payee_name']) ?></td>
<td><?= htmlspecialchars($acc['Bank_name']) ?></td>
```

// Each piece of user data is encoded before the browser displays it

By encoding all output this way, malicious input is displayed as harmless text rather than executing as code.

Cross-Site Request Forgery (CSRF) attacks occur when a malicious website tricks a user into submitting a form to our legitimate website without their knowledge. We prevent this by generating and validating unique tokens for each form. In every form, we include this token as a hidden field. When the form is submitted, it's verified with the token matches. Only requests originating from our website will have the correct CSRF token. Attacks from malicious websites cannot guess the random 64-character token.

```
// Generate CSRF token if one doesn't exist
if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));

    // bin2hex() converts random bytes into a readable hexadecimal string

    // random_bytes(32) generates 32 cryptographically random bytes

    // This creates a 64-character token unique to this user's session
}
```

Our application implements three-factor authentication requiring email, phone number, and password. All three credentials must be correct for login to succeed. Having this authentication, prevents other people from accessing a user's account.

```
function authenticate(PDO $pdo, string $email, string $phone, string $password): array|false {
    $stmt = $pdo->prepare(
        'SELECT * FROM Account_Holder WHERE Email = :email AND Phone_number = :phone'
```

```

);

// This query looks for a record where BOTH email AND phone are correct
// If either doesn't match, no record is returned

$stmt->execute([':email' => $email, ':phone' => $phone]);

$user = $stmt->fetch();

// fetch() returns the matching user record if found, or false if not found


// Now verify the password matches the stored hash
if ($user && password_verify($password, $user['Password_Hash'])) {
    return $user;

    // All three credentials match - authentication successful
}

return false;

// Email/phone not found or password incorrect
}

```

Within the login.php:

```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $email = trim($_POST['email'] ?? '');
    // trim() removes extra spaces from user input

    $phone = trim($_POST['phone'] ?? '');
    // ?? " provides a default empty string if data isn't present

    $password = $_POST['password'] ?? '';
    // Don't trim password (whitespace in passwords matters)


    $user = authenticate($pdo, $email, $phone, $password);
    // authenticate() checks if all three credentials are correct

```



```

if ($user) {

    login($user); // All three credentials matched

    session_regenerate_id(true); // Prevent session fixation attacks

    header('Location: index.php');

    exit;

} else {

    $error = 'Invalid email, phone number, or password.'; // Generic message

    // We don't reveal which credential was wrong

}

}

```

This three-factor approach is more secure than single-factor (password-only) authentication because an attacker would need all three pieces of information to gain access.

Passwords are never stored in plain text in the database, instead they are hashed using bcrypt, a cryptographic algorithm designed specifically for password security. Even if a hacker steals the database, they cannot recover the user password.

Why Bcrypt is Secure:

1. One-way encryption - The password hash cannot be reversed back to the original password. Even if a hacker steals the database, they cannot recover users' passwords.
2. Bcrypt automatically adds random "salt" to each password before hashing, so identical passwords produce different hashes. This prevents rainbow table attacks where attackers use pre-computed hashes.
3. Computationally expensive with cost 12, each password hash takes approximately 100 milliseconds to compute. This makes brute-force attacks (trying thousands of passwords) extremely slow and impractical.
4. Bcrypt's cost parameter can be increased as computers become faster, ensuring the algorithm remains secure over time.

Authorization ensures that each user can only view and modify their own data. We implement this by filtering all database queries with the User_ID parameter. Authorization is enforced at the database level through User_ID filtering, preventing users from accessing or modifying other users' data. We want to ensure that users feel that their information is protected at all times, and therefore can rely that the information, and amounts are accurate.

Session management maintains a secure authentication state for logged-in users. When authentication is successful, user information is stored in the session. After successful login, we regenerate the session ID to prevent session fixation attacks. Protected pages verify the user has a valid session before displaying content. Sessions are stored server-side, not in browser cookies, making them more secure. Sensitive data like passwords are never stored in session variables.

```
function login(array $user): void {  
  
    $_SESSION['user_id'] = $user['User_ID']; // Store user's ID  
  
    $_SESSION['first_name'] = $user['First_Name']; // Store first name  
  
    $_SESSION['last_name'] = $user['Last_Name']; // Store last name  
  
    $_SESSION['email'] = $user['Email']; // Store email  
  
    // Note: Password hash is NEVER stored in session for security  
  
}
```

Input validation ensures that user-submitted data is checked before use. We validate for empty fields, proper email format, and prevent duplicate accounts. Firstly there is registration validation, then email correctness, and then password strength validation. Finance Tracker is always checking to see if there is an already existing account. We use generic error messages that do not reveal which specific credential failed, preventing attackers from determining which email addresses exist in our system. Lastly, HTML5 input type validation enforces data format at the form level.

```
<input type="email" id="email" name="email" required>  
  
// type="email" validates that input looks like an email address  
  
<input type="tel" id="phone" name="phone" required>  
  
//type="tel" is for phone numbers  
  
<input type="password" id="password" name="password" minlength="8" required>  
  
//type="password" hides characters; minlength enforces minimum length
```

Our Finance Tracker implements industry-standard security practices at multiple layers. By using prepared statements to prevent SQL injection, output encoding to prevent XSS attacks, CSRF tokens to prevent cross-site request forgery, three-factor authentication with email and phone verification, bcrypt password hashing with cost 12 to protect passwords, User_ID filtering to restrict data access, session management for secure login state, and comprehensive input

validation, we created an application that effectively protects user financial data from common web vulnerabilities.

G. Optimization & Reliability Enhancements

To improve performance and reliability, our group added indexes to multiple columns in the Transactions table. Users may want to search transactions by account, date, or category, and financial changes should not be partially saved if one step fails.

The first index created was `idx_account_date`, which was added to the `Account_ID` and `date` columns in the Transactions table. This index is useful because many finance tracking queries search for transactions from a specific account within a certain date range. By indexing both the account ID and date together, it's easier to locate matching transactions more efficiently instead of searching the entire table.

The second index was `idx_category`, which was added to the `Category_ID` column in the Transactions table. This index improves queries that filter, search, or group transactions by category. Since Finance Tracker is meant to help users review and understand their spending habits, category based searches are important and frequently used.

```
1 CREATE INDEX idx_account_date
2 ON Transactions(Account_ID, date);
3
4 CREATE INDEX idx_category
5 ON Transactions(Category_ID);
```

One important query retrieved transactions for a specific user during a selected date range.

```
1 SELECT *
2 FROM Transactions t
3 INNER JOIN Account a ON t.Account_ID = a.Account_ID
4 WHERE a.User_ID = 50
5 AND t.date >= '2025-01-01'
6 AND t.date < '2026-01-01'
7 LIMIT 1000;
```

```

"table": {
  "table_name": "t",
  "access_type": "ref",
  "possible_keys": [
    "fk_account_id",
    "idx_account_date"
  ],
  "key": "fk_account_id",
  "used_key_parts": [
    "Account_ID"
  ],
  "rows_examined_per_scan": 1,
  "rows_produced_per_join": 0,
  "filtered": "11.11",
  "cost_info": {
    "read_cost": "1.53",
    "eval_cost": "0.03",
    "prefix_cost": "3.04",
    "data_read_per_join": "107"
  },
},

```

from the user's perspective because the transaction history pages can load faster.

One multi-step database operation occurs when a user adds a new transaction. The application first inserts the transaction into the Transactions table and then updates the related account balance in the Account table. This is needed because a transaction affects both the user's transaction history and the account's current balance.

The purpose of this query is to display transactions connected to a specific user during 2025. It joins the Transactions table with the Account table so the system can find all transactions connected to accounts owned by a specific user. This is important because users need to view their own financial activity by date, especially when reviewing yearly spending, income, or history.

It was optimized by using idx_account_date index. Since the query filters by transaction date and joins through Account_ID, the index helps MySQL narrow down relevant transaction records more efficiently. Without this index, the database may need to scan more rows in the Transactions table before finding the matching records. With the index, the database can better support searches involving account activity and date ranges, which are commonly used in our application.

The use of limiting results to 1000 also helps performance because it prevents the query from returning an unneeded large number of rows at once. This makes it more reliable

```

//Add transaction
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['add_transaction'])) {
    $stmt = $pdo->prepare(
        'INSERT INTO Transactions
        (Account_ID, Category_ID, Pay_ID, date, amount, description, transaction_type)
        VALUES
        (:acc, :cat, :pay, :date, :amount, :desc, :type)'
    );

    // Update account balance: Deposit adds, Withdrawal subtracts
    $type = $_POST['transaction_type'];
    $op = ($type === 'Deposit') ? '+' : '-';
    $bal = $pdo->prepare(
        "UPDATE Account SET Balance = Balance $op :amount WHERE Account_ID = :id"
    );
    $bal->execute([':amount' => $_POST['amount'], ':id' => $_POST['account_id']]);

    header('Location: index.php');
    exit;
}

```

If the transaction record is inserted but the account balance is not updated, the database would become inconsistent, if the account balance changes but the transaction is not saved, the user would lose the record detailing why the balance changed

H. Reflection & Final Improvements

The strengths highlighted in our Phase III feedback was the cleanliness and easy accessibility of our application which also ties into organization of our dataset. We did not want to make our user interface too cluttered, as it would decrease user interest, but also could make interacting with our database messy.

Each of our group members had different skills and strengths, so we were able to portion out the work each week and each member took initiative to work on the portions that they felt strongest in. Angel was the first person to take initiative with creating the database as well as the application. He placed the first foundational blocks that allowed the rest of us to go in and add in our own pieces as well. Jonathan played a huge role in organizing our database into 3NF form and optimization. He had a good eye for efficiency and organization of our data. Jeena was very strong with security and she played a role in helping us plan out how to make our database strong and protected. She also was one of our strongest presenters and allowed us to convey our ideas to the audience. Keanna played a huge role in integrating our database with our application and creating a design that would be good for the eye, while keeping it clean. She aided Angel in the application portion of our presentation. Aila kept the group organized, making sure that there were weekly checks and meetings in order to keep good time management and communication. She also was in charge of a lot of the documentation throughout the semester and created the algorithm that allowed us to filter our data into different categories.

One of the biggest challenges throughout the semester was being able to find time for all of us to meet and work cohesively. We all had different schedules, which would cause someone to be unable to join a meeting here and there. However, everyone stayed very active in the chat and was able to communicate their work and what was done. This helped make up for some of the missed meetings.

We used the ToyStory database as a skeleton and example for our application in order to get it started, so we have to spend time nitpicking at what we felt did not go with our vision. It was very helpful to have a general application started, so this made altering our application for our finance database easier.

If we had more time, we would have liked to find a way to incorporate automatic updates to the database. For example, our database heavily relies on manual input. This can cause inaccurate data because not every transaction would be inserted into the database. We could achieve this through real-time API integration with actual banks to automatically pull transactions and update balances. Additionally, we would have liked to incorporate some sort of "recommendation" aspect where the application can examine what the user's weaknesses are in terms of finance and could recommend a solution. This could include advanced analytics features

such as spending trend charts, budget alerts, and year-over-year analysis to help users identify financial patterns and areas for improvement.

Beyond financial recommendations, we would have liked to enhance our security infrastructure by implementing audit logging to track all login attempts, failed authentications, and suspicious activity for both security monitoring and compliance. Rate limiting and account lockout mechanisms would prevent brute-force attacks and automatically send security alerts to users after multiple failed login attempts. Additionally, add data export features allowing users to generate and download transaction reports as CSV or PDF files, along with monthly automatic backups for data protection and recovery options.

Finally, if we had time we could implement recurring transaction templates to auto-fill commonly repeated transactions such as rent and utilities, which would reduce manual data entry errors. SMS-based two-factor authentication could provide an additional security layer beyond our current bcrypt password hashing, allowing users to verify logins through their phone. Enhanced transaction tagging and note functionality would allow users to add custom tags and detailed notes to transactions, improving organization beyond our current Category system. These improvements would transform Finance Tracker into a more robust, secure, and user-friendly application that reduces manual work while providing deeper financial insights.

Database Application Deployment

Your database application must be deployed and accessible online via a live URL. You may use cPanel to host your application, following the same process used in previous assignments:

- ☐ Complete [Create cPanel Deployment File](#) to create the file needed to deploy your GitHub repository to cPanel. Make sure to save it in the root of your project folder.
- ☐ Upload your changes to GitHub. Refer back to [Uploading Changes \(GitHub Desktop\)](#) OR [Uploading Changes \(Git CLI\)](#) depending on which one you're using.

- ☐ Complete the [Clone GitHub Repository In cPanel section](#) to clone and deploy your GitHub repository in cPanel. Make sure to replace **repoName** with your database application name.
- ☐ Complete the [Pull Latest Version Of GitHub Repository In cPanel section](#) to pull the updates in your GitHub repository into cPanel.
- ☐ Complete the [Viewing Your Live Database Application section](#) to view your live database website and make sure it appears the way you intended. You can refer to the [Troubleshooting & Debugging section](#) if you are having trouble viewing your live web page.

You may also deploy your application using another platform. If you are using Node, Express, or React, you will not be able to deploy on cPanel. In this case, follow the alternative instructions provided in [Issues Hosting Node Web Apps on cPanel](#) to deploy your application using Render (or another supported platform).

Database Application Expectations

Your database application must:

- Be accessible online via a working URL
- Function reliably without major errors
- Demonstrate clear frontend–backend–database integration
- Represent your group’s final and polished work

Submission

When you're finished, complete the following steps to submit your work:

- ☐ Export your **project report** as a **PDF file** AND save it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:
 - [Google Docs](#) (File → Download → PDF Document)
 - [Microsoft Word](#) (File → Save As / Export → PDF)
 - [Pages](#) (File → Export To → PDF)
- ☐ Export your **final database** as an **SQL file** into the **db** folder. Follow the steps in the [Exporting A Database section](#) to export your database into a single SQL file.
- ☐ Commit and merge **final changes** to your **source code** into the **main** branch
- ☐ Upload all your changes to GitHub.
 - ☐ **If you're using GitHub Desktop (GUI)**, complete the [Uploading Changes \(GitHub Desktop\) section](#) to upload your changes from your local device to GitHub.
 - ☐ **If you're using Git (CLI)**, complete the [Uploading Changes \(GitHub CLI\) section](#) to upload your changes from your local device to GitHub.

ONE group member must paste the **URLs of your GitHub repository AND live database application** in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.